

mopi: Local Semantic Search

Bridging the Gap Between Speed and Intelligence

Project Presentation

1. Introduction & Problem Statement

- **The Latency Trap:** Cloud-based semantic search (OpenAI, etc.) introduces network overhead (500ms+), breaking the "search-as-you-type" experience.
- **The Privacy Gap:** Traditional local search is keyword-only. Modern semantic search often requires uploading private files to third-party servers.
- **The Intelligence Ceiling:** Lexical search (grep, etc.) fails on synonyms and conceptual intent.

2. Project Objectives

- **Local-First:** All processing, from embedding to indexing, must happen on the user's silicon.
- **Interactive Performance:** Achieve sub-50ms P95 latency for hybrid queries.
- **Format Agnostic:** Support Plain Text, Code, PDF, and Office documents.
- **Modular Stability:** A decoupled architecture with a central daemon (`mopid`) and lightweight clients.

3. Proposed Solution & Innovation

- **Hybrid Retrieval Engine:** Combines the precision of Tantivy (BM25) with the recall of Vector Search (HNSW).
- **Reciprocal Rank Fusion (RRF):** A mathematically robust way to merge disparate search scores into a single ranked list.
- **Semantic Chunking:** Unlike naive fixed-size splitting, mopi understands file structure (paragraphs, headings) to preserve context.

4. Methodology (Technology Stack)

- **Language:** Rust (Performance, Memory Safety).
- **Deep Learning:** ONNX Runtime via **FastEmbed-RS**.
- **Lexical Index:** Tantivy.
- **Metadata:** SQLite.
- **Client/Server:** Unix Domain Sockets (IPC) with length-delimited binary framing.

5. System Architecture / Workflow

1. **Crawl:** Discover files, handle symlinks, and detect changes.
2. **Extract:** Convert raw bytes into normalized text and metadata.
3. **Embed:** Generate vectors using quantized local models.
4. **Index:** Synchronize SQLite metadata, Tantivy documents, and ANN vectors.
5. **Query:** Client sends intent -> Daemon runs hybrid search -> Streams results.

6. Implementation

- **Model Choice:** `all-MiniLM-L6-v2` (384 dimensions).
- **Optimization:**
 - **Quantization:** `INT8` weights for 4x size reduction.
 - **Hardware Acceleration:** AVX-512 and SIMD utilization.
- **Concurrency:** Dedicated worker pools for extraction and embedding to prevent blocking the query path.

7. Results / Outcomes

- **Latency:**
 - Query Embedding: ~15ms
 - Search & Fusion: ~15ms
 - Total Interactive Loop: ~35ms
- **UX:** Enabled a launcher-style GUI (kiwi) that updates semantically as the user types.
- **Accuracy:** Significant recall improvement over keyword-only systems for conceptual queries (e.g., "how to setup auth" finding README.md).

8. Conclusion

- **mopi** proves that high-quality semantic search does not require the cloud.
- By optimizing the inference stack and using hybrid ranking, we provide a search experience that is both private and faster than remote alternatives.
- **Future:** Integration of cross-encoders for reranking and hardware-specific kernels (Metal/CUDA).

9. References

- **FastEmbed-RS:** <https://github.com/qdrant/fastembed>
- **Tantivy:** <https://github.com/quickwit-oss/tantivy>
- **Marp CLI:** <https://marp.app/>
- **ONNX Runtime:** <https://onnxruntime.ai/>